Instituto Infnet

Quem responde, promete; quem falha, explica

Projeto de Bloco: Inteligência Artificial Aplicada

Aulas 13–14 · Etapa 7 (versão compacta) · Respostas e erros — TP4, itens 2, 3 e 4

Prof. Renan Santos

Onde paramos (Aula 12)

A v10 RECEBE relatos — e um modelo Pydantic decide o que entra.

- **O bilhete: 'o modelo do POST do SEU projeto — e um campo que você vai recusar'**
  Quem recusou um dado pessoal? Quem ainda tem um campo dict?
- **Quem recebe, valida**
  O 422 com a lista inteira; extra='forbid'; o painel que traduz.
- **A ponte que ficou: o /docs mostrava 'string' em toda resposta 200**
  A API sabia dizer o que ACEITA. Não sabia dizer o que DEVOLVE.

▸ **Hoje a saída ganha contrato — e o modelo que conferia a entrada passa a conferir a resposta.**

🎬 O que a API devolve — e o que ela diz quando falha

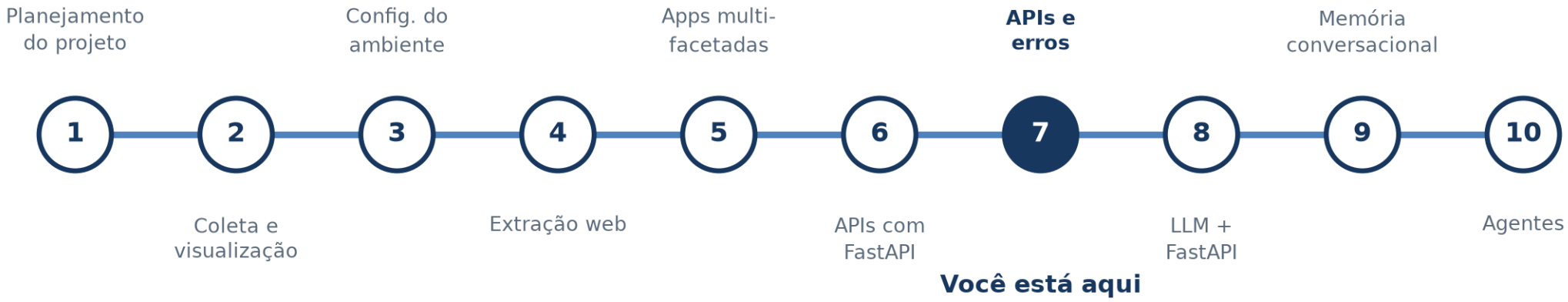
Três pedidos, em poucas semanas.

- **O técnico da prefeitura: "o /docs diz que toda resposta é string"**
  Ele montou o código olhando o que voltava — e quebrou no /fontes: cada item com campos diferentes.
- **A gestora: "a prefeitura quer saber JÁ o formato da análise do LLM"**
  O LLM é a Etapa 8.
- **E o relatório da prefeitura: "2026 — nenhum desmatamento registrado"**
  O PRODES nem publicou 2026. A API respondeu 200, 'linhas': 0.

▸ **A pergunta da aula: o que a API promete devolver — e o que ela diz quando dá errado?**

Onde estamos no Projeto de Bloco

Etapa 7 inteira, hoje: a resposta e o erro também são contrato.



Objetivos da aula

Quatro capacidades — o contrato do que sai, e do que sai quando dá errado.

- **Declarar a resposta — e diagnosticar a promessa quebrada**
  response_model: filtra, confere, documenta; o 500.
- **Projetar o contrato do POST /processar_texto**
  Entrada e saída prontas antes do motor (o LLM).
- **Levantar a exceção certa — e proteger a API**
  HTTPException: 404, 409, 503; a rede de segurança do 500.
- **Fazer o painel ler pelo status**
  E transformar cada falha em feedback útil.

O que esta aula entrega ao TP4

A Etapa 7 inteira: subcompetências 4.1 e 4.2.

- **Item 2: "POST /processar_texto: rota que recebe um texto… e retorna a análise"**
  A rota de pé, com um motor provisório (regras).
- **Item 3: "Response Models… o texto processado, o tipo de análise… os resultados"**
  ResultadoAnalise — e um modelo de resposta em toda rota.
- **Item 4: "exceções HTTP específicas… 503… feedback útil para o usuário final"**
  404, 409, 503 com HTTPException; o 500 com código; o painel que lê pelo status.

▸ **Depois de hoje: itens 2, 3 e 4. O LLM (itens 1–2) ganha as duas semanas antes da entrega.**

1 · Quem responde, promete

🐛 O /docs que dizia 'string'

Passo 1, ao vivo — a v10 por dentro. Abram cada rota e rolem até 'Responses'.

- **Resposta 200 de TODAS as rotas: 'string'**
  No /openapi.json, o esquema vazio — nas 9 rotas da v10.
- **/fontes: três itens com 11, 6 e 3 campos · /noticias: 'palavras': '431' (texto)**
  O JSON de cada coleta, inteiro — até os segundos que a coleta do PRODES levou.
- **/processar_texto: 8 campos — 2 são detalhes do motor**
  'pontuacoes' e 'texto_normalizado' saem junto.

> ▸ **Quem responde, promete. A v10 não prometia nada sobre a saída — e o /docs não tinha o que mostrar.**

O response_model: o contrato da saída

passo02_response_model.py

```
class ResultadoAnalise(BaseModel):
    tipo_analise: Literal["classificacao"]
    texto: str
    categoria_sugerida: Categoria | None
    confianca: float = Field(ge=0, le=1)
    pistas: list[str]
    motor: str

@app.post("/processar_texto", response_model=ResultadoAnalise)
def processar_texto(pedido: PedidoAnalise):   # a MESMA função
```

Filtra, confere, documenta

As funções do passo 2 são as mesmas do passo 1. A resposta muda — de três jeitos.

- **FILTRA — só o que está no modelo sai**
  /processar_texto: 8 → 6 campos. /fontes: 11 · 6 · 3 → 5 · 5 · 5. (No livro: o id some [p64–65].)
- **CONFERE — o que sai passa pelo modelo**
  'palavras': '431' → 431 (o modelo diz int). O que não converte… é o passo 3.
- **DOCUMENTA — o /docs mostra os campos**
  E os Schemas, no fim da página. Nada de 'string'.

▸ **O livro mostra o resultado (o id que some) e não explica o mecanismo. O mecanismo são esses três verbos.**

🐛 A promessa quebrada (passo 3)

```
terminal do uvicorn  ·  GET /relatos-v1  ->  500

# o cliente vê:   Internal Server Error   (texto puro)

fastapi.exceptions.ResponseValidationError: 6 validation errors:
  {'type': 'extra_forbidden', 'loc': ('response', 0, 'id'),
   'msg': 'Extra inputs are not permitted', 'input': 1}

# GET /noticias-v1  (publicado_em: datetime)  ->  500
  {'type': 'datetime_from_date_parsing',
   'msg': '… invalid character in year', 'input': '02/09/2026 08:16'}
```

422 × 500 — e um contrato para cada direção

Os dois são o Pydantic recusando. A diferença é de quem é a culpa.

- **422 (Aula 12): o CLIENTE mandou errado · 500 (hoje): a API respondeu errado**
  loc 'body' × loc 'response'. O motivo do 500 só aparece no TERMINAL.
- **O modelo de ENTRADA (Relato, forbid) na resposta → 500**
  A linha gravada tem id e recebido_em, que a entrada proíbe. A saída é RelatoGravado.
- **Prometa só o que o seu dado cumpre**
  publicado_em: str ('dd/mm/aaaa hh:mm') — e não datetime.

▸ **Um contrato para cada direção: o relato que entra não tem número; o que sai, tem.**

🐛 O 201 que fez o cliente dizer 'falhou'

status_code=status.HTTP_201_CREATED: criou, 201 [p68] — e mudou o contrato.

- **Os relatos estavam gravados; o programa dizia FALHOU**
  Ele conferia r.status_code == 200.
- **O nosso painel não quebrou**
  raise_for_status() confere a FAMÍLIA (tudo abaixo de 400 passa) — desde a Aula 2.
- **No cliente: r.ok (ou 200 <= status < 300)**
  'The first digit of a status code defines its category.' [p62]

▸ **Mudar a resposta é mudar o contrato. A API avisa; o cliente confere a família, não o número.**

Livro × hoje — o capítulo 3 inteiro

O livro usa o Pydantic 1, mostra efeitos sem o mecanismo — e deixa lacunas.

- **class Config: schema_extra [p63–64] · typing.List [p63] · -> dict + response_model [p64]**
  Hoje: ConfigDict(json_schema_extra=…) · list[...] · vale o response_model.
- **O id que some [p64–65] sem explicação · o 404 'Undocumented' [p68] sem comentário**
  O mecanismo: filtra, confere, documenta. A documentação do erro: responses=.
- **Não cobre: o 500 de validação, 422, 503, exception_handler, responses=**
  Busca no PDF inteiro: zero. Vêm da documentação oficial — verificados por execução.

▸ **Sexta vez no semestre que a fonte está anterior. A pergunta é sempre: o que roda hoje?**

2 · O contrato antes do motor

A rota do TP4: POST /processar_texto

api/rotas/analise.py (e passo05_contrato_do_motor.py)

```
class PedidoAnalise(BaseModel):              # o que ENTRA (Aula 12)
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    texto: str = Field(min_length=15, max_length=2000)
    tipo_analise: Literal["classificacao"] = "classificacao"

@router.post("/processar_texto", response_model=ResultadoAnalise)
def processar_texto(pedido: PedidoAnalise):
    resultado = classificar(pedido.texto, REGRAS)   # o MOTOR
    return {"tipo_analise": ..., "texto": ..., "motor": MOTOR, **resultado}
```

Os campos do TP4 — e o limite do motor

'…o texto processado, o tipo de análise realizada… e os resultados gerados.' — TP4, item 3

- **texto · tipo_analise · categoria_sugerida (ou null) · confianca · pistas · motor**
  O que o motor calcula a mais (as pontuações) não está no contrato: um LLM não as teria.
- **'A água do arroio subiu… alcançou as casas' → null, 0.0, []**
  Uma enchente — e nenhuma palavra da lista. O argumento da Etapa 8, medido no nosso dado.
- **A página 🔎 Analisador mostra só os campos do contrato**
  Uma linha no menu.

▸ **O contrato vem antes do motor. Na Etapa 8, troca-se só classificar().**

3 · Quem falha, explica

🐛 O zero que não era zero

'retrieving a to-do that doesn't exist returns a 200 response status code instead of a 404' [Adeshina, p66]

- **GET /v11/ranking/2026 → 200, {'linhas': 0, 'dados': []}**
  O sistema da prefeitura somou as linhas — zero — e escreveu 'nenhum desmatamento'.
- **GET /ranking/2026 → 404: 'O PRODES não tem a taxa de 2026: a série vai de 2022 a 2025.'**
  O status diz que não existe; a frase diz o que existe.
- **'A 200 status code shouldn't be returned for a failed request operation.' [p62]**
  O programa decide o que fazer ANTES de ler o corpo — e a API tinha dito 'deu certo'.

▸ **O status é a primeira palavra da resposta. 200 para o que falhou é a mentira mais cara.**

HTTPException: levantar o erro

passo01_200_ou_404.py

```
@app.get("/ranking/{ano}")
def ranking(ano: int):
    dados = [...]
    if not dados:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"O PRODES não tem a taxa de {ano}: a série vai de 2022 a 2025.",
        )
    return {...}            # o cliente recebe: 404 {"detail": "O PRODES…"}
```

Não existe ≠ vazio

O critério da Aula 11, de volta: o caminho IDENTIFICA; a consulta RECORTA.

- **/ranking/2026 · /relatos/99 → 404**
  O caminho identifica UM recurso — e ele não existe.
- **/relatos?uf=AC (nenhum do Acre) → 200, 'linhas': 0**
  A consulta recorta uma coleção — 'nenhum' é a resposta certa para 'quais?'.
- **/desmatamento?desde=2030 → 200, 'linhas': 0**
  Idem: o período recorta. Não é falha.

▸ **404 para o filtro vazio obriga o cliente a tratar como erro uma pergunta bem respondida.**

422 × 404 × 409

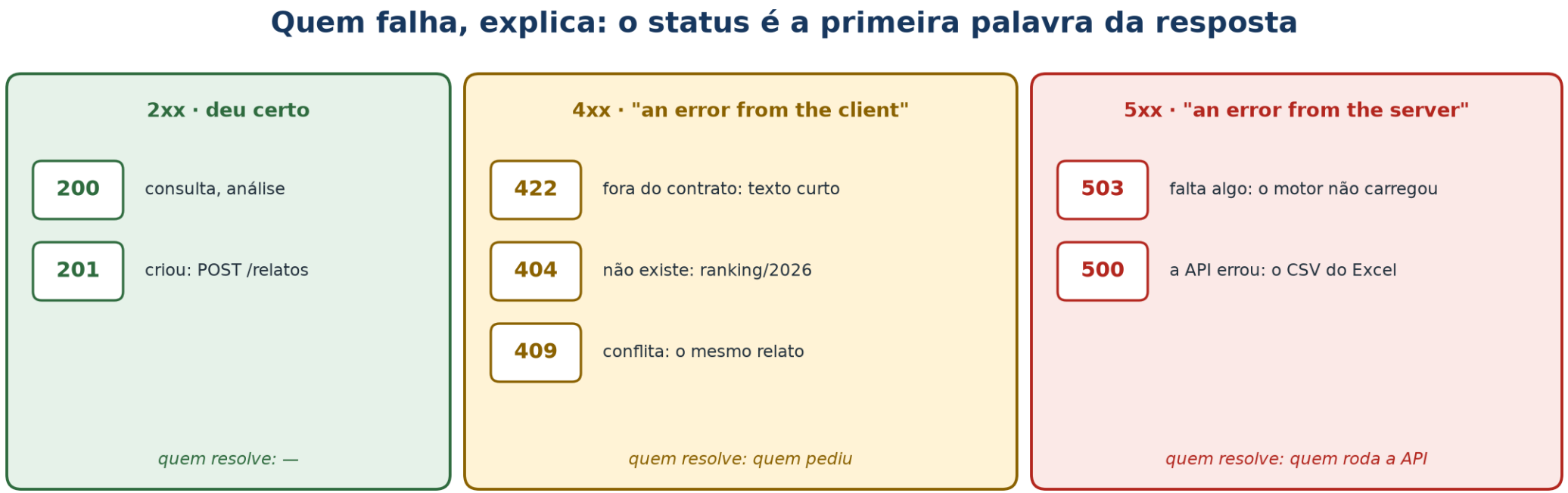
Três 4xx, três coisas diferentes (o passo 2 da Aula 14 fica para casa).

- **POST novo → 201 · o MESMO de novo → 409: 'Este relato já foi recebido: nº 1, em …'**
  O corpo está certo — o conflito é com o ESTADO. O modelo não tem como saber; quem guarda sabe.
- **texto 'fumaça' → 422 (string_too_short) · /relatos/99 → 404**
  422: o pedido fora do contrato. 404: o recurso não existe.
- **No projeto: RelatoRepetido em src/relatos.py — a rota traduz para 409**
  src/ fala a língua do domínio; a API traduz para HTTP. (O livro: 409 no cadastro repetido [p96].)

▸ **Três 4xx, três coisas diferentes para o cliente fazer: corrigir, pedir outra coisa — ou nada.**

O mapa dos status

Cada falha do projeto tem um status — e cada status diz quem resolve.



> **"A 400 status code shouldn't be returned for a server error. Likewise, a 200 status code shouldn't be returned for a failed request operation." [Adeshina, p62]**

O motor com try — e o 503

api/rotas/analise.py (e passo03_motor_fora.py)

```
try:
    REGRAS = carregar_regras()          # na Etapa 8: carregar o modelo LLM
except (OSError, ValueError) as falha:
    REGRAS = None
    log.error("Analisador NÃO carregado — %s", falha)   # o motivo: no log

@router.post("/processar_texto", responses={503: erro("…")})
def processar_texto(pedido: PedidoAnalise):
    if REGRAS is None:
        raise HTTPException(503, detail="O analisador está indisponível…")
```

🐛 A API que não subia

Tropeço real da v11 — e o exemplo do próprio TP4.

- **Sem regras.json (ou com uma vírgula a mais): FileNotFoundError ao importar**
  Nenhuma rota respondia — nem o desmatamento, nem os relatos.
- **Na v12: a API sobe; /processar_texto → 503; o resto → 200**
  'O analisador está indisponível… As outras rotas funcionam normalmente.'
- **E sem a coleta: v11 → 500 (ValueError: min() iterable argument is empty); v12 → 503**
  'Os dados do PRODES não foram coletados nesta máquina. Rode a coleta…'

▸ **Falhe em pedaços, não inteira. Na Etapa 8, o motor é um modelo de centenas de MB — vai falhar mais.**

🐛 O Excel e o 500 em texto puro

Tropeço real da v12: alguém abriu relatos.csv no Excel e salvou — com ';' no lugar de ','.

- **GET /relatos → KeyError: 'id' → 500, content-type text/plain: 'Internal Server Error'**
  Um programa que fazia r.json() quebrou: JSONDecodeError.
- **O painel v11 mostrou os 5 relatos FICTÍCIOS no lugar dos reais**
  A explicação — 'a API falhou (HTTPError)' — ficou numa legenda.
- **O livro cita o 500 só como 'internal server error' [p62]**
  Sem tratamento. É o passo 4.

▸ **Você não vai prever tudo. Precisa de uma rede para o que não previu.**

A rede de segurança

api/erros.py (e passo04_erro_inesperado.py)

```
def erro_inesperado(request: Request, exc: Exception) -> JSONResponse:
    codigo = uuid.uuid4().hex[:8]
    log.error("Erro inesperado [%s] em %s %s — %s: %s", codigo,
              request.method, request.url.path, type(exc).__name__, exc)
    return JSONResponse(status_code=500, content={
        "detail": f"Erro interno na API (código {codigo}). Informe o código…"})

app.add_exception_handler(Exception, erro_inesperado)
# terminal: Erro inesperado [b5d0ecd4] em GET /relatos — KeyError: 'id'  + o traceback
```

4 · O painel lê pelo status

🐛 O cliente ingênuo — e o 'Tente de novo'

Dois tropeços reais do lado de quem LÊ a resposta.

- **'Para cada item do detail, mostre o msg' → TypeError no primeiro 404**
  O detail do 422 é uma LISTA; o dos outros, uma FRASE — percorrida letra por letra.
- **O painel da Aula 13 a um 409: 'A API não respondeu (HTTPError). Tente de novo.'**
  A API tinha respondido — e o 'tente de novo' levava a outro 409.
- **E a um 500: os relatos fictícios de exemplo, como se fossem os recebidos**
  A explicação, numa legenda.

▸ **Só a API sabe o que aconteceu. O status decide como ler o corpo — e o painel mostra o que ela DISSE.**

O status decide como ler o corpo

src/cliente_api.py · _falha() e FalhaDaAPI

```
def _falha(r):
    if r.status_code == 422:                                  # uma LISTA
        return FalhaDaAPI(422, [traduzir(e) for e in r.json()["detail"]])
    try:
        frase = r.json()["detail"]                            # uma FRASE
    except (ValueError, KeyError, TypeError):
        frase = f"A API respondeu {r.status_code} sem explicar o motivo."
    return FalhaDaAPI(r.status_code, [frase])

# as páginas: except FalhaDaAPI as f: (st.warning se 409/503, senão st.error)
```

O tom: aviso ou erro

Não é só o texto: é a cor. O usuário precisa saber se tem algo a corrigir.

- **409 → AVISO: 'Este relato já foi recebido: nº 3…'**
  Está gravado. Não há o que corrigir — nem o que repetir.
- **503 → AVISO + um link para o que funciona**
  'O analisador está indisponível… As outras rotas funcionam.'
- **422 · 500 → ERRO**
  O texto curto (corrija); o erro interno, com o código (avise quem mantém).
- **Lista com erro → a tela DIZ que mostra exemplos**
  Na v11, isso estava numa legenda.

A21-23 · seis jeitos de a API mentir

A API de antes de hoje. Para cada um: o STATUS, o que o outro lado recebe, QUEBRA ou ENGANA, e o que corrige.

- 1. GET /relatos com o modelo de ENTRADA na resposta · 2. o cliente == 200 depois do 201
- 3. ranking/2026 (o PRODES não publicou) · 4. o mesmo relato, duas vezes
- 5. o clone sem regras.json: uvicorn api.main:app · 6. o CSV salvo pelo Excel, na página Relatos

Ao vivo: a v12

terminal · na pasta ../aula-14/demo/painel_ods_brasil/

```
uvicorn api.main:app --reload        # terminal 1 — a API
streamlit run app.py                 # terminal 2 — o painel

# /docs: POST /processar_texto → os seis campos · ranking/2026 → 404
# 🔎 Analisador: o rio → seca  ·  🗣️ Relatos: um envio, depois o MESMO → aviso 409
# renomeie data/analisador/regras.json, salve um .py → o aviso do 503 (desfaça!)

git commit -am "v12: response models, POST /processar_texto, 404/409/503 e o 500 em JSON"
git push
```

O TP4, depois de hoje — a Etapa 7 completa

O enunciado, item por item.

- ✅ **Item 2: POST /processar_texto de pé**
  O pedido validado; a análise na saída.
- ✅ **Item 3: ResultadoAnalise — e um modelo de resposta em toda rota**
  Respostas estruturadas e validadas.
- ✅ **Item 4: 404, 409, 503 do modelo; o 500 com código; o painel que lê pelo status**
  'Feedback útil para o usuário final' e 'facilitando o diagnóstico'.
- 🔜 **Etapa 8: o modelo LLM local, no lugar de classificar()**
  O contrato, o try com o 503 e o timeout já prontos.

▸ **Entrega do TP4: semana de 16–21/11 — e a Etapa 8 ganhou as duas semanas antes dela.**

Anti-padrões — da resposta e do erro

Quase todos passam no teste de quem escreveu a API.

- **Rota sem modelo de resposta · o modelo de entrada na saída · datetime para 'dd/mm/aaaa'**
  O /docs diz 'string'; 500 na rota inteira, com o motivo só no terminal.
- **Conferir == 200 · 200 para o que falhou · 404 para o filtro vazio · 422 para o conflito**
  'FALHOU' a um 201; o zero que vira dado; o erro que não é erro.
- **O modelo carregado sem try · o 500 em texto puro · o traceback no detail**
  A API inteira cai; r.json() quebra; o caminho do arquivo vaza.
- **O cliente que trata todo erro como 422 · 'tente de novo' a um 409**
  A tela que inventa a resposta.

Princípios que levamos

Para o TP4 — e para qualquer API.

- **Quem responde, promete · um contrato para cada direção**
  E prometa só o que o seu dado cumpre.
- **O contrato vem antes do motor**
  O LLM da Etapa 8 entra atrás de uma rota que já existe.
- **Quem falha, explica · o status é a primeira palavra · não existe ≠ vazio**
  404 × 200 [].
- **Falhe em pedaços · ao usuário, a frase; ao log, o traceback**
  E o status decide como ler o corpo.

🎬 O caso, em 3 lições (v11–v12)

O que a Etapa 7 ensinou, na ordem em que doeu.

- **Quem responde, promete**
  O /docs com 'string'; o modelo de entrada que derrubou a saída.
- **O status é a primeira palavra**
  O 200 vazio virou 'nenhum desmatamento em 2026'.
- **Falhe em pedaços — e ao usuário, a frase**
  A API que não subia; o 'tente de novo' a um 409.

▸ **Etapa 8: um modelo de linguagem atrás de uma rota que já sabe responder — e falhar.**